PAPERNEXUS: A Local-First Research Harness for Paper-Centered Knowledge Work

PaperNexus Project

Abstract

PAPERNEXUS is a local-first research harness for paper-centered knowledge work, designed for the case where a research assistant can produce fluent synthesis while losing the provenance of the papers, claims, methods, quotes, access decisions, and evaluation outcomes that justify it. The system turns topics, PDFs, and Markdown sources into durable research memory: discovery records with legal full-text status, reusable semantic snapshots, typed graph entities, method-evolution evidence, graph-backed answers, benchmark reports, and authenticated human or agent interfaces. This report evaluates the current implementation as an operational research system rather than as a new retrieval model. On May 12, 2026, public-benchmark audits on a 64-logical-CPU Linux server completed sub-1GB runs and a bounded sub-5GB expansion. The fixed-corpus expansion ran TREC-COVID, SciDocs, LitSearch full, and ScholarGym with a per-run fixed-corpus index cache; LitSearch full completed 597 queries over 64,183 papers in 2:44.21, while ScholarGym required a 16GB Node heap for 2,458 queries over 570,204 papers. A STaRK-MAG diagnostic imported 1,872,968 nodes and 19,919,698 edges into Kuzu and improved Hit@100 from 0.345 for lexical-only seeds to 0.393 after 1–2 hop graph expansion, but with 4.77s average query latency. Local operational audits further showed that heuristic-only 100-PDF upload and import completed in 30.289 seconds without failed import tasks, whereas LLM-enriched paths were dominated by provider latency and required bounded batching. The contribution is therefore a provenance-preserving, inspectable research workflow with measured execution surfaces, not a leaderboard claim.

1 Introduction

Academic research increasingly depends on long chains of machine assistance: search engines retrieve candidate papers, PDF parsers extract text, language models summarize sections, graph tools connect ideas, and agents turn intermediate notes into research plans. The practical bottleneck is no longer only whether a system can find a paper. It is whether the system can preserve enough structure for a later user to ask: Where did this claim come from? Which method improved which predecessor? Was the cited paper actually available as an open PDF? Which part of the corpus supports this cross-domain analogy?

PAPERNEXUS is built around the conservative assumption that evidence becomes fragile when research memory is represented only as files, embeddings, or prompt-time summaries. The system therefore treats the research graph as the primary artifact. Papers are not only documents to retrieve; they are sources of typed objects such as problems, methods, claims, evidence, datasets, benchmarks, takeaways, idea fragments, and future directions. Literature discovery is not only a download helper; it is a record of what was searched, merged, resolved, skipped, and legally available. Research answers are not only generated prose; they are views over an evidence-bearing graph.

This paper follows the style of functional technical reports for research harnesses such as ARIS [20]: it emphasizes the operating assumptions, architectural layers, workflow surfaces, assurance mechanisms,

evaluation hooks, and deployment footprint of the system. We intentionally de-emphasize line-by-line implementation details. The goal is to explain what a researcher can do with PAPERNEXUS, how the pieces fit together, and why the system is organized around local ownership, explicit evidence, and measurable behavior.

2 Functional Thesis

PAPERNEXUS is designed around four linked claims about paper-centered automation. A literature review does not end when a set of snippets is retrieved; the researcher needs to revisit earlier parses, compare method families, inspect unresolved sources, and ask new questions without repeating the whole pipeline. The system therefore stores raw sources, markdown cache, semantic snapshots, graph artifacts, queue state, discovery outputs, and derived overlays as local, reusable assets. This durable-memory stance is the basis for the later interface and evaluation design, because every downstream answer or benchmark should be traceable to a preserved intermediate state.

For paper-centered work, the most useful automation surface is often not a chatbot response but a structured memory that can support many later responses. PAPERNEXUS places graph construction before answer construction, so question answering, method lineage, cross-domain search, and idea generation operate over committed graph state rather than relying on query-time invention. This ordering does not remove language models from the workflow; instead, it confines them to enrichment and presentation steps whose inputs and outputs can be inspected. The result is a system in which generated prose is treated as a view over evidence, not as the evidence itself.

The system also assumes that automation must remain inspectable across human and agent workflows. PAPERNEXUS exposes state through CLI commands for operators, a browser dashboard for inspection, HTTP APIs for applications, MCP tools for agents, and Python scripts for remote workflows. These interfaces are deliberately different views of the same local research memory rather than independent silos. That design matters because the same corpus may be curated by a human researcher, checked by a script, queried by a coding agent, and audited later through saved run artifacts.

Finally, evaluation and secrets are treated as first-class operational concerns rather than release afterthoughts. A research harness that discovers papers, calls external providers, and serves remote agents must be measurable, recoverable, and sanitizable before it is shared. PAPERNEXUS therefore includes retrieval-benchmark loaders and metrics, encrypted local environment storage, bearer-token guarded server surfaces, queue recovery, and a release-hygiene path that removes personal paths, accounts, local state, and generated build traces from public artifacts. These mechanisms do not make the system scientifically complete, but they define the baseline for making its behavior inspectable enough to improve.

3 System Overview

Figure 1 summarizes the functional architecture. The left side introduces sources, the center maintains durable research memory, and the right side exposes human and agent workflows.

3.1 Layered Responsibilities

The separation is practical. Discovery can be repeated without rebuilding the graph. Parsing can be retried without losing the source manifest. Graph construction can be staged and committed after inspection. Interfaces can query committed state without re-running expensive upstream work.

4 Functional Workflows

4.1 Topic to Corpus

The discovery workflow starts from a topic, seed question, related-work paragraph, venue, author, dataset, or manually supplied paper. It plans complementary queries, sends them to open metadata providers, merges candidates, expands through citations when available, and resolves legal full-text sources. It uses services such as OpenAlex [11], Semantic Scholar [13], Crossref [6], arXiv [2], DBLP [7], Europe PMC [8], CORE [5], and Unpaywall [12].

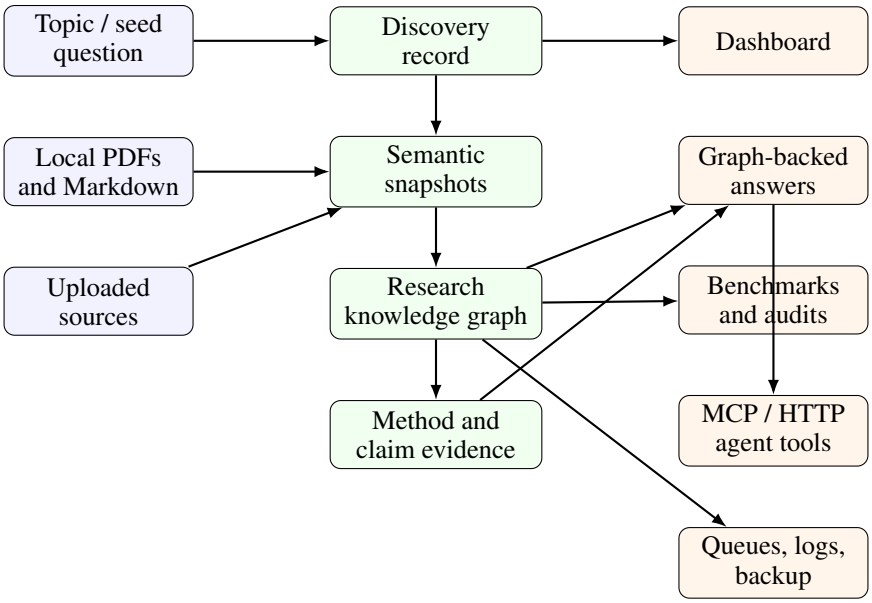

Figure 1: Functional view of PAPERNEXUS. Discovery, ingestion, graph memory, evidence, benchmark, agent, and operations layers are maintained as local artifacts or controlled interfaces without changing the local ownership model.

Table 1: Functional layers in PAPERNEXUS.

Layer	Role in the research workflow
Discovery	Turns a topic into provider queries, merged candidates, open-access status, download manifests, and importable sources.
Ingestion	Converts local PDFs or Markdown into reusable paper snapshots with parser metadata and semantic extraction results.
Graph memory	Represents papers, problems, methods, claims, evidence, datasets, benchmarks, takeaways, ideas, and relations as a persistent local graph.
Evidence	Tracks method evolution, citation contexts, exact quotes, temporal direction, confidence, and blocked or ambiguous relationships.
Interfaces	Provides CLI, Web UI, HTTP API, local MCP, remote HTTP MCP, services, logs, and Python wrappers.
Evaluation and hygiene	Measures retrieval and task behavior, stores encrypted local secrets, and sanitizes release artifacts before publication.

The important output is not merely a folder of PDFs. The workflow records what was found and why a candidate could or could not enter the corpus. A result may be a resolved open PDF, a metadata-only paper, an item requiring institutional access, an access-control-blocked page, or a URL that returned HTML instead of a PDF. This distinction matters because a research assistant should not confuse "not downloaded" with "not relevant" or with "not legally available." The current implementation persists discovery runs with JSON artifacts, Markdown reports, download manifests, import-task links, candidate provenance, and latest-run pointers.

4.2 Corpus to Research Memory

Once sources exist locally, PAPERNEXUS materializes them into reusable semantic snapshots. The default path parses PDF or Markdown, stores markdown cache and metadata, and allows later LLM-assisted enrichment to run as a separate stage. This gives the user a durable intermediate representation: if graph construction changes, the system can reuse prior parsing rather than starting from raw PDFs.

The staged pipeline is intentionally simple:

```
sources -> materialize -> llm-optimize -> build-graph
        -> merge-graph -> write-index
```

The key property is resumability. A user can run the full analysis once, continue after interruption, rebuild only dirty stages, or inspect stage outputs before committing a new graph.

4.3 Research Memory to Evidence

The graph layer makes paper-centered objects queryable. Problems, methods, claims, evidence, datasets, benchmarks, takeaways, idea fragments, and future directions become typed nodes. Relations between them become explicit edges. This enables both conventional literature exploration and higher-level reasoning tasks.

Method evolution is treated as a special evidence problem. A system should not casually claim that one method extends, improves, replaces, adapts, or uses another method. PAPERNEXUS therefore records method aliases, citation contexts, quote evidence, candidate relationships, accepted relationships, stubs, and diagnostics. Strong method-evolution relationships require support beyond name overlap: the relationship must be grounded in context, direction, and confidence.

4.4 Evidence to Answers

Research-intelligence workflows read from graph state. They can answer cross-domain questions, produce method lineages, inspect evidence for a method relationship, return a method registry, or combine these views into a unified answer. The operating invariant is that these workflows should expose existing graph evidence rather than silently generate new evidence at query time.

This gives the system a useful division of labor. Language models may help enrich snapshots or phrase responses, but the evidence substrate remains an inspectable local graph. When a researcher asks for a mechanism, lineage, or analogy, the answer can point back to the corpus state that supports it.

4.5 Research Memory to Evaluation

PAPERNEXUS now includes a retrieval-benchmark path that can evaluate the discovery and retrieval surface rather than only describe it qualitatively. The benchmark command supports closed-corpus and live-discovery modes. Closed-corpus mode follows information-retrieval semantics: the system searches a provided corpus and is scored against query relevance labels. Live mode runs the normal provider-backed discovery workflow and scores returned papers against gold identifiers or titles.

The loader surface covers custom JSON/JSONL, BEIR-style corpora and TREC qrels, LitSearch-style query/corpus exports, BioASQ question files, native TREC biomedical topics plus qrels, CSFCube faceted annotations, and flexible schemas for SAGE, ScholarQABench/OpenScholar, PaperAsk, SPARBench, and SciNetBench. The reported metrics include hit@k, exact match@k, recall@k, weighted recall@k, precision@k, F1@k, MRR@k, MAP@k, and NDCG@k. For task-style benchmarks, PAPERNEXUS can additionally score answer overlap, citation precision and recall, claim-label accuracy, relation/path overlap, and optional LLM-judged answer grounding.

5 Interface Surface

PAPERNEXUS exposes the same research memory through several interfaces (Table 2). This is a functional choice rather than a convenience layer: different users need different levels of control over the same corpus.

The MCP surface follows the Model Context Protocol [10]. Its purpose is to make research memory available to agents without asking those agents to understand the internal storage layout. Important tool families include literature discovery, research lookup, research briefing, idea catalyst, and import workflow.

Table 2: Interface surfaces and their intended users.

Interface	Typical use
CLI	Initialize a corpus, run staged analysis, inspect status, search the graph, benchmark retrieval, manage secure environment values, export backups, and manage services.
Dashboard	Inspect corpus status, graph summaries, overlays, imports, runtime configuration, and human-readable views.
HTTP API	Integrate browser and server workflows with authenticated routes for graph, method-evidence, discovery, and research-answer queries.
Local MCP	Give a local coding or research agent tool access over stdio.
Remote HTTP MCP	Expose the same agent tool surface from a running server with bearer-token authentication.
Python wrappers	Submit imports, inspect queues, query graph state, and orchestrate remote workflows from scripts.
Secure environment	Keep local provider keys and runtime secrets in an encrypted file whose encryption key is held by the system keychain.

6 Assurance Mechanisms

PAPERNEXUS does not claim that every extracted relationship is correct. Instead, it makes uncertainty and evidence boundaries visible.

The first assurance boundary is access and parsing provenance. Discovery records whether full text is open, unavailable, restricted, blocked, or malformed, which prevents downstream tools from treating access failure as absence of evidence. Parsing and semantic extraction are separated from graph commit, so a failed parser, degenerate title, or incomplete snapshot can be detected before it pollutes the graph. This staging is deliberately conservative: it preserves imperfect intermediate artifacts as diagnostics while preventing them from being silently promoted into claims.

The second assurance boundary is graph and method-evidence validation. The graph schema constrains the kinds of objects and relationships that can be represented, while lite projections and metadata make committed state inspectable without requiring the full graph engine for every read. Strong method-evolution edges require resolved methods, citation context, quote support, confidence, temporal direction, and non-conflicting evidence. Ambiguous aliases and stubs are preserved as diagnostics rather than promoted as validated relationships, which reduces false precision in method-lineage queries.

The third assurance boundary is operational recovery and release hygiene. Queues, logs, backups, and staged commands allow the user to recover from interruptions and audit long-running work, especially when imports, PDF parsing, and LLM enrichment run over large corpora. Retrieval benchmarks separate fixed-corpus scores from live-provider scores so that provider availability, source resolution, and corpus retrieval quality remain distinct measurements. Local credentials can be loaded from encrypted secure-env storage, and public release artifacts can be sanitized so that personal account names, absolute home paths, local caches, bearer-token fixtures, and LaTeX build traces do not leak into documentation or arXiv source bundles.

7 Functional Footprint

Table 3 summarizes the current feature footprint without tying the description to source files.

8 Example Use Patterns

The most direct use pattern is literature mapping. A researcher starts with a topic such as "graph-augmented experiment planning," and PAPERNEXUS searches open metadata providers, merges duplicate candidates, records which papers have legal full-text access, imports resolved PDFs, and builds a graph that supports later search over problems, methods, claims, evidence, and datasets. The value of the workflow is not only that it finds candidate papers, but that it records the access and

Table 3: Current functional footprint.

Function	Current behavior
Literature discovery	Plans queries, merges multi-provider candidates, resolves legal full text, records coverage, and bridges resolved sources into the import queue.
PDF and Markdown ingestion	Supports local corpora and uploaded import tasks with parser configuration, fallback parsing, markdown cache, and semantic snapshots.
Graph construction	Builds a typed research graph and commits a lightweight read projection for query, dashboard, and agent workflows.
Method evolution	Builds method registries, validates method-to-method evidence, records candidates and accepted relationships, and exposes lineage/evidence lookups.
Research answers	Provides graph-backed cross-domain evidence, method lineage, method evidence, method registry, and unified answer modes.
Retrieval evaluation	Runs fixed-corpus and live-discovery retrieval benchmarks over custom, BEIR-style, biomedical, literature-search, and graph-reasoning benchmark formats.
Security hygiene	Provides encrypted local secure-env storage, bearer-token authenticated server surfaces, portable path helpers, and sanitized release artifacts.
Operations	Supports services, logs, dashboard serving, authenticated remote MCP, backup export, backup unpack, and queue inspection.

provenance decisions that would otherwise disappear into a download folder. That record lets the researcher distinguish a missing source from an inaccessible source, a metadata-only candidate from a parsed paper, and a graph-backed claim from an unsupported synthesis.

A second use pattern is method lineage review. A researcher asks how a method family evolved, and the system searches validated method-to-method relationships rather than relying only on name overlap or citation adjacency. It reports predecessor and successor links together with the citation contexts and quote evidence that support the relationship. This makes the difference between a substantive improvement claim and a mere background comparison visible to the user, which is essential when method lineage later informs a related-work section or a new research idea.

A third use pattern is agent-assisted research and benchmarking. A coding or research agent can connect to local or remote MCP and call tools for discovery status, graph lookup, research answers, paper index lookup, import workflow, and idea-catalyst operations without scraping folders directly. The same corpus can then be evaluated through fixed-corpus or live-discovery retrieval benchmarks. In fixed-corpus mode, PAPERNEXUS reads a benchmark corpus, query set, and qrels, then reports ranking metrics; in live mode, the same benchmark query can exercise OpenAlex, Semantic Scholar, Crossref, arXiv, and configured optional providers. This separation makes external-provider behavior measurable without conflating it with closed-corpus retrieval quality.

9 Evaluation and Deployment Observations

The measured result is operational rather than competitive: the current PAPERNEXUS harness can run small public retrieval audits, local import stress tests, queue recovery checks, and qwen-based LLM diagnostics, but it is not yet a repeated-run benchmark result suite. The directly comparable fixed-corpus anchors show that the observed SciFact and NFCorpus rows trail published BEIR BM25 results, so the retrieval layer should be treated as an improvement target rather than as a leaderboard claim. The deployment evidence is stronger for provenance and recoverability: discovery feeds ingestion, ingestion feeds graph construction, method evidence attaches to graph memory, and intelligence workflows consume committed graph state through CLI, dashboard, HTTP, MCP, and script interfaces. This section separates public benchmark audits, sub-5GB expansion measurements, external-framework comparability, and local pipeline timing so that each claim remains tied to the measurements that support it.

For the local verification run used by this report, the project passed the full Node test suite and a release-hygiene scan. The test suite passed 366 of 366 tests. The scan found no repository-text, arXiv-source, or PDF string matches for private home paths, personal account names, long bearer-token literals, OpenAI-style keys, GitHub-style tokens, or private-key blocks.

9.1 Sub-1GB Public Benchmark Measurements

On May 12, 2026, we ran a small public-benchmark exercise on a remote server with constrained local storage. The run was intentionally scoped to datasets below 1GB and did not write to the production Kuzu graph database. New datasets and run artifacts were placed under `/data1`; the existing `/data2` filesystem was already full. The purpose was not to claim leaderboard comparability, but to audit whether the current retrieval harness, live-provider path, and qwen-based task evaluation path can execute on small public tasks.

The remote benchmark host, recorded as jin-Super-Server at 10.126.56.41, ran Linux 6.8.0-90-generic on x86_64. The environment audit recorded an AMD EPYC 7542 32-Core Processor, 64 logical CPUs, 125GiB RAM, Python 3.10.16 in the `ccd` environment, and a project-local Node `v22.16.0` runtime. The PAPERNEXUS checkout used for the run was under `/data1/hyq/PaperNexus`. At run time, `/data1` had 15T total capacity with about 3.3T available, while `/data2` was already full enough that new datasets, run outputs, and discovery caches were written under `/data1`. The qwen rows called an OpenAI-compatible `cliproxyapi` provider; provider-side hardware was not visible in PAPERNEXUS logs.

Table 4 is therefore reported as a statistical-readiness table rather than as a formal ML comparison. Each PAPERNEXUS row is a single observed execution ($n = 1$); no value is presented as a mean over repeated runs, no 95% confidence interval is estimable, no paired significance test is valid, and no effect size is reported. The table also includes paper-reported external anchors from the most relevant benchmark papers where the task and metric can be stated precisely. These anchors are not treated as head-to-head significance tests.

The fixed-corpus observations in Table 4 support an engineering claim, not a statistical one: the benchmark harness can execute SciFact, NFCorpus, CSFCube, and a LitSearch sample [1] under the tested storage constraints. Against the directly comparable BEIR nDCG@10 anchors, the observed PAPERNEXUS SciFact and NFCorpus rows are below published BM25 and BM25+CE results [16]. That gap should be treated as a regression target for the retrieval harness rather than as an inferential result, because the current data contain no repeated runs and no paired per-query significance test. The LitSearch row is more realistic for literature retrieval, but the comparison is only directional until PAPERNEXUS reports LitSearch's official broad R@20 and specific R@5/R@20 slices. In the single LitSearch sample100 run, the 64,183-paper corpus took 23:12.54 wall-clock time and 2.27GB peak RSS. This made it a useful bottleneck diagnostic and motivated the fixed-corpus index path used for the full LitSearch run reported in Table 8.

The live ScholarQABench observations [4, 3] are likewise diagnostic rather than official. In the retrieval-only row of Table 4, 24 provider failures occurred across 10 queries: 21 arXiv HTTP 429 responses and 3 Semantic Scholar HTTP 429 responses. The LLM-enhanced row used the local OpenAI-compatible qwen configuration with 90-second LLM timeouts and enabled discovery request caching. It generated 10 system answers and 10 qwen judge calls with no malformed JSON in this sample. The full row still took 15:32.32 wall-clock time, or about 93.2 seconds per query, and the citation F1 of 0.300 matched the retrieval recall@10 of 0.300. The practical implication is that live-provider caching, provider backoff, query-level checkpointing, and bounded batched LLM answer/judge calls are prerequisites before these measurements can be promoted into repeated statistical experiments.

9.2 Sub-5GB Benchmark Expansion Measurements

The next expansion kept the same storage discipline but raised the ceiling from sub-1GB to sub-5GB public artifacts. Tables 5 and 6 record the task selection and metric plan that guided the run. Tables 7, 8, 9, 10, and 11 report the resulting single executions. These are still statistical-readiness measurements: each row has $n = 1$, so no standard deviation, 95% confidence interval, paired significance test, or effect size is claimed.

Table 4: Statistical-readiness audit for the May 12, 2026 sub-1GB benchmark exercise, with external paper-reported anchors. External values are included only where their source, task boundary, and metric can be stated; they are not repeated-run comparisons against PAPERNEXUS.

Benchmark	PAPERNEXUS setting	PAPERNEXUS single observed value	External reported result	Comparison status
SciFact	fixed; $n = 1$; 300 q / 5.2k p	nDCG@10 0.617; R@10 0.739; MRR@10 0.583	BEIR reports nDCG@10 0.665 for BM25 and **0.688** for BM25+CE on SciFact [16].	Same BEIR dataset family and same nDCG@10 metric; the single PAPERNEXUS run is below the published BEIR BM25 and BM25+CE anchors.
CSFCube	fixed; $n = 1$; 50 q / 4.2k p	nDCG@10 0.162; R@10 0.016; MRR@10 0.180	No directly matching result was found in the currently cited BEIR, LitSearch, OpenScholar, PaSa, or SPAR benchmark papers.	Diagnostic row only; keep it for faceted-retrieval regression, not for external leaderboard comparison.
NFCorpus	fixed; $n = 1$; 323 q / 3.6k p	nDCG@10 0.283; R@10 0.133; MRR@10 0.484	BEIR reports nDCG@10 0.325 for BM25 and **0.350** for BM25+CE on NFCorpus [16].	Same BEIR dataset family and same nDCG@10 metric; the single PAPERNEXUS run is below the published BEIR anchors.
LitSearch sample100	fixed; $n = 1$; 100 / 597 q; 64k p	nDCG@10 0.360; R@10 0.503; MRR@10 0.330	LitSearch reports BM25 Avg. broad R@20 39.9 and specific R@5 50.0; GritLM-7B 70.8 / 74.8; GPT-4o reranking over GritLM **75.3 / 79.2** [1].	Same 64,183-paper corpus, but PAPERNEXUS used a 100-query sample and reports R@10/nDCG/MRR, while LitSearch reports broad R@20 and specific R@5/R@20. Directional only until official LitSearch metrics are emitted.
ScholarQABench live10	live retrieval; $n = 1$; 10 / 208 q	R@10 0.300; MRR@10 0.167; 24 provider failures	OpenScholar reports answer-level ScholarQABench metrics rather than a directly matching live-provider R@10 row [3].	Diagnostic live-provider row; not directly comparable to OpenScholar because the latter uses OSDS and evaluates synthesis/citation quality.
ScholarQABench live10 + LLM	live + qwen; $n = 1$; 10 / 208 q	task success 0.720; answer correctness 0.720; citation F1 0.300	On Scholar-CS, OpenScholar reports Rubric/Cite-F1 of 51.1/47.9 for OpenScholar-8B, 57.7/39.5 for OpenScholar-GPT-4o, 45.6/48.0 for PaperQA2, and 45.0/0.1 for GPT-4o without retrieval [3].	Same benchmark family but not the same setting: PAPERNEXUS used a 10-query live-provider sample and qwen judging, while OpenScholar uses its 45M-paper OSDS and official rubric/citation evaluation.

The executed sequence followed this plan: TREC-COVID and SciDocs broadened fixed-corpus regression; LitSearch full ran after adding a per-run fixed-corpus index and candidate prefilter; ScholarGym tested a static deep-research corpus; ScholarSearch and AutoResearchBench were kept as live-provider diagnostics; and STaRK-MAG was isolated in a scratch Kuzu database. BioASQ, OpenScholar's OSDS, PaperSearchQA PubMed, full S2ORC, and SciNetBench remain outside this tier unless the server has a separate high-capacity scratch allocation.

The fixed-corpus rows in Tables 7 and 8 show that the new index path changes the operational profile. The earlier LitSearch sample100 run took 23:12.54 for 100 queries because it repeatedly scanned the corpus. The full LitSearch run completed all 597 queries in 2:44.21 after the in-memory corpus index and inverted candidate prefilter were enabled. The improvement should be treated as an engineering result, not a model-quality claim, because the scoring protocol still does not emit LitSearch's official broad R@20 and specific R@5/R@20 slices. ScholarGym is the current memory ceiling: it completed, but only after raising the Node heap to 16GB, and it still had a 0.421 zero-match rate under the 5,000-document scan limit.

The STaRK-MAG rows in Tables 10 and 11 answer a different question from the fixed-corpus rows: whether Kuzu can carry a graph-retrieval diagnostic at this scale. The answer is yes for import and bounded traversal, but not yet for quality. The graph-expanded run raised Hit@100 and Recall@100, while Hit@10 stayed unchanged at 0.202. This means the current lexical seed selection and graph scoring place additional relevant papers into the candidate set but do not rank them early enough. The next graph-retrieval work should add relation-aware seed parsing, high-degree node penalties, path-

Table 5: Sub-5GB benchmark expansion candidates used for the May 12, 2026 execution. The table defines task boundaries; measured values are reported in the following result tables.

Candidate	Public footprint	Comparable task	Why it belongs in the sub-5GB tier
LitSearch full	About 2.85GB repository	Fixed-corpus scientific literature search over 64,183 papers and 597 queries [1].	Most direct full-scale fixed-corpus literature-search comparison.
BEIR TREC-COVID	About 111MB	Biomedical COVID literature retrieval with a larger corpus than SciFact/NFCorpus [16].	Adds large-corpus biomedical IR without leaving BEIR metrics.
BEIR SciDocs	About 19MB	Citation-oriented scientific-document retrieval [16].	Tests paper-to-paper retrieval rather than fact-verification only.
ScholarGym	About 861MB	Static deep-research information gathering over roughly 570k papers [14].	Strongest static alternative to unstable live-provider academic search.
ScholarSearch / AutoResearchBench	Query/gold artifacts are small	Live scholarly search and agentic literature-discovery diagnostics [21, 19].	Measures provider behavior, resume, and multi-step search traces.
STaRK-MAG	Preflight required for MAG KB	Text-plus-graph retrieval over academic entities and relations [17].	Only useful sub-5GB candidate that stresses Kuzu graph retrieval directly.

Table 6: Metrics and implementation work for the sub-5GB expansion candidates. This table defines what to emit before any new result can be treated as comparable.

Target class	Primary metrics	Required implementation work
LitSearch full	Official broad R@20 and specific R@5/R@20; also nDCG@10, MRR@10, latency.	Add broad/specific split reporting and persistent fixed-corpus index caching before the full 597-query run.
BEIR expansion	nDCG@10, Recall@50/100, MAP@10, MRR@10, index time, peak RSS.	Reuse BEIR loader; add memory guard and separate index-build time from query-evaluation time.
Static deep-research	Gold-paper Recall@20/50/100, paper-set F1, source coverage, trace success.	Add ScholarGym adapter, identity normalization, query checkpoints, and optional agent trace logging.
Live agentic search	Source Recall@20/50, Precision/F1@20/50, provider failures, retries, cache hit rate, resume success.	Add loaders for query/gold files; persist query rewrites, provider responses, cache hashes, and partial results.
Graph retrieval	Hit@K/Recall@K, text-only versus graph-only versus hybrid scores, relation/path evidence coverage.	Import into a bench-scratch Kuzu database, keep production graph isolated, and add graph-retrieval ablations.

type features, and a text-only versus graph-only versus hybrid ablation before claiming comparability to official STaRK results.

9.3 Comparability to External Frameworks

PAPERNEXUS is closest to academic-search and paper-centered research-agent systems, but the level of comparability depends on the task boundary. The safest presentation is to separate closed-corpus retrieval, literature-search retrieval, live academic paper search, citation-grounded QA, and method-graph evaluation. Tables 12, 13, 14, 15, and 16 therefore split the comparison into task-specific views. These tables do not introduce repeated-run statistics: all PAPERNEXUS values remain single observed runs, and all external values are copied from the cited papers as reported anchors.

The strongest current claim is therefore not that PAPERNEXUS outperforms these systems. The defensible claim is that it now has the substrate required to compare against them in layers without erasing task boundaries. The direct BEIR comparison in Table 12 identifies an immediate retrieval regression target: the observed SciFact and NFCorpus nDCG@10 values are below published BM25 anchors, while the newer TREC-COVID and SciDocs rows in Table 8 extend that regression surface. The LitSearch comparison in Table 13 is directionally stronger after the full 597-query run, but it is still incomplete until the official broad and specific slices are emitted. The ScholarQABench comparison in Table 14 shows that retrieval recall and citation quality should be reported separately. The PaSa/SPAR comparison in Table 15 requires same-query live paper-set evaluation with provider traces, identity normalization, cache hashes, and date-stamped provider behavior. The method-graph comparison in Table 16 requires gold method-evolution fixtures before any claim about graph quality. AutoResearchBench, ScholarSearch, and STaRK-MAG now have diagnostic rows in this report, but they remain non-official comparisons until query rewriting, gold normalization, and official metric

Table 7: Single-run sub-5GB fixed-corpus runtime on the 41 server. All rows used fixed-corpus mode, `-max-candidates 1000`, and `PAPERNEXUS_FIXED_CORPUS_SCAN_LIMIT=5000`. Values are not repeated-run means.

Dataset	Queries	Papers	Wall / peak RSS	Runtime note
BEIR TREC-COVID [16]	50	171,332	8:54.15 / 2.56GB	Large qrels; useful for large-corpus latency regression.
BEIR SciDocs [16]	1,000	25,657	6:38.92 / 1.19GB	Citation-oriented paper-to-paper retrieval.
LitSearch full [1]	597	64,183	2:44.21 / 1.47GB	Full run after enabling the fixed-corpus index cache.
ScholarGym [14]	2,458	570,204	29:32.76 / 10.25GB	Required `NODE_OPTIONS=-max-old-space-size=16384`.

Table 8: Single-run sub-5GB fixed-corpus retrieval metrics. Top-10 columns report nDCG@10, Recall@10, and MRR@10; top-100 columns report Hit@100, Recall@100, and nDCG@100.

Dataset	nDCG10	R10	MRR10	Hit100	R100	nDCG100	Pool	Zero
BEIR TREC-COVID [16]	0.557	0.014	0.870	1.000	0.090	0.381	0.337	0.000
BEIR SciDocs [16]	0.153	0.157	0.274	0.763	0.336	0.214	0.541	0.073
LitSearch full [1]	0.374	0.510	0.334	0.730	0.719	0.417	0.853	0.139
ScholarGym [14]	0.142	0.189	0.147	0.428	0.330	0.174	0.466	0.421

adapters are implemented. AstaBench, LiveDRBench, and SciNetBench remain future diagnostic targets.

9.4 Local Pipeline Measurements

On May 11, 2026, we ran several local measurements to separate graph commit, synchronization, PDF parsing, and LLM-based semantic enrichment latency. The tests were run against local PAPERNEXUS corpora and authenticated local server surfaces. Tables 17, 18, 19, 20, and 21 report these as operational audit tables. They are not public benchmark comparisons: each condition has one independent execution unless explicitly stated, so the tables do not report between-run standard deviations, confidence intervals, exact p-values, or effect sizes. Their role is to identify bottlenecks and failure modes that must be fixed before the public benchmark rows can be turned into repeated experiments.

These operational measurements were run from the project workstation rather than the 41 benchmark host. The environment audit recorded macOS 15.1 on `arm64`, 10 logical CPUs, 16GB RAM, and Node `v25.4.0`. The upload and queue tests used authenticated local server routes; the qwen batch tests used the same local OpenAI-compatible `cliproxyapi` configuration, with provider-side hardware again unavailable to local logs.

The upload stress audit in Table 21 used the HTTP import route with base64 file payloads, the background import worker, the MarkItDown PDF parser, four-way analysis concurrency, and `semanticExtraction=heuristic-only`. Therefore the measured path includes upload, queueing, PDF-to-Markdown conversion, materialization, the pipeline's LLM-optimization stage in heuristic-only mode, and fast graph commit, but excludes external LLM enrichment calls. The 100-PDF condition completed without failed import tasks in its single run. Its additional cumulative substage timings were 0.418 seconds for Markdown reads, 0.499 seconds for Markdown parsing, and 1.665 seconds for semantic snapshots. Because parsing ran concurrently, 113.589 seconds of cumulative materialization work completed in 30.289 seconds of wall time, or roughly 3.3 uploaded PDFs per second for this small-PDF workload. This throughput is a practical observation, not a statistically significant estimate.

Taken together, these audit tables support cautious engineering conclusions. The current graph write and synchronization path was not the observed bottleneck in the tested runs. Without external LLM enrichment, PDF-to-Markdown conversion dominated the import path. With LLM enrichment enabled, the LLM-optimization stage dominated graph commit in both the single-import timing logs and the aggregate 10-paper pipeline log. The queue-recovery audit also validates the intended continue behavior: stale historical failures can be reconciled without deleting task logs or uploaded sources, while the active queue presents the recovered state to agents. The CPA/Qwen batch audit shows why the system should not optimize by sending arbitrarily large paper batches to a single prompt: the 10-paper condition worked once, while the 100-paper condition returned HTTP 200 with unusable truncated JSON. The safer production design is prompt-budget batching, deterministic

Table 9: Single-run live and agentic diagnostic results. These rows intentionally measure provider and query-shaping behavior, not closed-corpus retrieval quality.

Diagnostic	Scale	Wall / latency	Observed result and failure mode
AutoResearchBench sample [19]	40 q; 175 gold papers	5:24.77; 8,110.7ms/query	Hit@10 0.000; R@10 0.000; zero-match 1.000; 58 OpenAlex HTTP 400 failures and 5 Semantic Scholar HTTP 429 failures.
ScholarSearch diagnostic [21]	50 q; no usable gold papers	4:14.39; 5,082.9ms/query	Avg. retrieved count 0.26; no retrieval metric is meaningful without gold papers; intended max-10 run evaluated 50 queries.

Table 10: STaRK-MAG Kuzu import diagnostic. The database was built from the sub-5GB MAG artifact in a scratch run directory and did not touch the production PAPERNEXUS graph.

Stage	Scale	Wall / peak RSS	Main observed values
CSV preparation	1,872,968 nodes; 19,919,698 edges	1:02.87 / 6.91GB	Wrote node and edge import files plus 84 query seed lists.
Kuzu COPY import	same full graph	0:10.24 / 2.35GB	Imported all nodes and edges; scratch artifact footprint 1.4GB.

schema validation, batch-level checkpoints, and split retry/continue for malformed or oversized outputs.

This conclusion is consistent with the staged extraction design studied from Intern-Atlas [18]. That system does not rely on one giant prompt over full papers. It parses PDFs into structured sections using Nougat for born-digital PDFs and GROBID as a fallback, then runs a two-phase LLM extraction process over selected evidence. The first phase uses Introduction and Method sections plus reference titles and abstracts, averaging about 21.2 references and 5.8k input / 2.1k output tokens per prompt. The second phase only processes non-background candidates, batches about 10 candidates at a time, averages about 7.9k input / 2.8k output tokens, and repairs malformed batches once before dropping persistent failures. For PAPERNEXUS, the practical implication is to keep full-text parsing local and reusable, feed the LLM only the sections needed for the target extraction, and make every LLM batch independently resumable.

10 Limitations and Future Work

PAPERNEXUS has several current limitations that matter for interpreting the reported measurements. Metadata-only discovery candidates are recorded but are not automatically promoted into full graph paper nodes unless they pass through import and materialization. Discovery source coverage remains incomplete: OpenReview, Papers with Code, DataCite, OpenAIRE, BASE, HAL, Zenodo, bioRxiv, medRxiv, SSRN, OSF, and Chinese scholarship providers are natural extensions. Systematic-review-style screening is not yet a standalone workflow with include/exclude tables and reviewer decisions, and version relations such as preprint-of, extended-by, artifact-for, and dataset-for are not yet fully represented as graph relations. Method-evolution quality also depends on extraction quality; conservative validation reduces false positives but may miss valid relationships that are weakly expressed in source text.

The evaluation limitations are equally important. The benchmark layer has loaders and metrics, but the current public numbers are single executions rather than repeated experiments. The sub-1GB and sub-5GB runs show that the harness can execute fixed-corpus retrieval, live retrieval, qwen-based task evaluation, static deep-research retrieval, and scratch Kuzu graph diagnostics under the tested storage constraints, but they do not establish statistical significance or leaderboard-level superiority. Future evaluation should therefore focus on discovery recall against systematic-review gold sets, open-access status accuracy, parser success and metadata accuracy, graph entity and relation

Table 11: STaRK-MAG Kuzu retrieval diagnostic over 84 human-eval queries. Retrieval used title/name lexical seeds plus undirected 1–2 hop Kuzu expansion.

Method	Hit10	Hit100	R100	nDCG100	MRR	Latency
Lexical-only seeds	0.202	0.345	0.202	0.136	0.153	included
Kuzu graph-expanded	0.202	0.393	0.255	0.146	0.155	avg 4.77s; p95 6.87s

Table 12: Direct fixed-corpus retrieval anchors on BEIR-family tasks. The gap column is a descriptive arithmetic difference between the single PAPERNEXUS run and the paper-reported BEIR BM25 anchor, not a significance test.

Dataset	Metric	PAPERNEXUS	BEIR BM25	BEIR BM25+CE	Interpretation
SciFact	nDCG@10	0.617	0.665	**0.688**	Direct metric and dataset-family match; PAPERNEXUS is 0.048 below BEIR BM25 in this single run [16].
NFCorpus	nDCG@10	0.283	0.325	**0.350**	Direct metric and dataset-family match; PAPERNEXUS is 0.042 below BEIR BM25 in this single run [16].

precision, method-edge precision, fixed-corpus retrieval quality, live-provider retrieval quality, answer faithfulness, and official metric adapters for LitSearch, ScholarGym, STaRK-MAG, ScholarSearch, and AutoResearchBench. These measurements should be paired with engineering checks for latency, throughput, queue recovery, service stability, and secure release hygiene.

11 Conclusion

PAPERNEXUS is a local-first research harness for turning paper-centered work into durable, inspectable research memory. Its main contribution is not a single retrieval model or a single agent prompt, but the integration of discovery, ingestion, graph construction, method-evidence validation, graph-backed research intelligence, benchmark execution, and multi-surface operation into one local workflow. The current evidence supports a constrained but useful conclusion: the system can preserve provenance across sources, snapshots, graph state, agent interfaces, queue recovery, and preliminary benchmark audits, while its retrieval quality and LLM-enriched paths still require repeated evaluation and targeted optimization. PAPERNEXUS is therefore best understood as research infrastructure for automation with provenance, where generation is downstream of memory, evidence, and measurement.

Acknowledgments

This report was generated from the local PAPERNEXUS working tree and project documentation on May 12, 2026. The writing style was revised to follow the functional technical-report pattern exemplified by ARIS [20]; no text from that paper is reused.

References

[1] Anirudh Ajith, Mengzhou Xia, Alexis Chevalier, Tanya Goyal, Danqi Chen, and Tianyu Gao. Litsearch: A retrieval benchmark for scientific literature search. https://arxiv.org/abs/2407.18940, 2024.

[2] arXiv. arxiv api user manual. https://info.arxiv.org/help/api/, 2026.

[3] Akari Asai, Jacqueline He, Rulin Shao, Weijia Shi, Amanpreet Singh, Joseph Chee Chang, Kyle Lo, Luca Soldaini, Sergey Feldman, Mike D'arcy, David Wadden, Matt Latzke, Minyang Tian, Pan Ji, Shengyan Liu, Hao Tong, Bohao Wu, Yanyu Xiong, Luke Zettlemoyer, Graham Neubig, Dan Weld, Doug Downey, Wen-tau Yih, Pang Wei Koh, and Hannaneh Hajishirzi. Openscholar: Synthesizing scientific literature with retrieval-augmented language models. https://arxiv.org/abs/2411.14199, 2024.

[4] Akari Asai, Jacqueline He, Rulin Shao, Weijia Shi, Amanpreet Singh, Joseph Chee Chang, Kyle Lo, Luca Soldaini, Sergey Feldman, Mike D'arcy, David Wadden, Matt Latzke, Minyang

Table 13: Directional LitSearch comparison. LitSearch reports broad-query R@20 and specific-query R@5 slices; the current PAPERNEXUS full run reports all 597 queries with R@10, nDCG@10, and MRR@10, so no head-to-head rank is claimed until the same broad/specific slices are emitted.

System or setting	Reported metrics	Comparison boundary
PAPERNEXUS full LitSearch	R@10 0.510; nDCG@10 0.374; MRR@10 0.334; R@100 0.719	Same 64,183-paper corpus and all 597 queries, but still different metrics from the paper's broad/specific slices.
LitSearch BM25	Broad R@20 39.9; specific R@5 50.0	Paper-reported lexical baseline [1].
LitSearch GritLM-7B	Broad R@20 70.8; specific R@5 74.8	Paper-reported dense baseline [1].
LitSearch GPT-4o rerank over GritLM	**Broad R@20 75.3**; **specific R@5 79.2**	Direct comparison requires PAPERNEXUS to emit the same broad/specific slices [1].

Table 14: Citation-grounded QA anchors. The PAPERNEXUS row is a 10-query live-provider diagnostic with qwen answer and judge calls; OpenScholar rows use the paper's Scholar-CS evaluation setting and are not head-to-head reproductions.

System or setting	Retrieval store	Correctness-like metric	Citation-like metric	Interpretation
PAPERNEXUS live10 + qwen	Live OpenAlex/S2/arXiv	Answer correctness 0.720	Citation F1 0.300	Diagnostic only; supporting-paper R@10 was also 0.300, limiting citation quality.
OpenScholar-8B	OSDS 45M papers	Rubric 51.1	Cite-F1 47.9	Paper-reported Scholar-CS anchor [3].
OpenScholar-GPT-4o	OSDS 45M papers	**Rubric 57.7**	Cite-F1 39.5	Paper-reported stronger answer-quality anchor with lower Cite-F1 than OpenScholar-8B [3].
PaperQA2	Retrieval-augmented setting	Rubric 45.6	**Cite-F1 48.0**	Paper-reported baseline in the OpenScholar comparison [3].
GPT-4o without retrieval	No retrieval store	Rubric 45.0	Cite-F1 0.1	Shows why citation-grounded tasks need retrieval rather than pure generation [3].

Tian, Pan Ji, Shengyan Liu, Hao Tong, Bohao Wu, Yanyu Xiong, Luke Zettlemoyer, Graham Neubig, Dan Weld, Doug Downey, Wen-tau Yih, Pang Wei Koh, and Hannaneh Hajishirzi. Scholarqabench dataset repository. https://github.com/AkariAsai/ScholarQABench, 2024.

[5] CORE. Core api documentation. https://core.ac.uk/services/api, 2026.

[6] Crossref. Crossref rest api. https://api.crossref.org/, 2026.

[7] DBLP. Dblp api documentation. https://dblp.org/faq/How+to+use+the+dblp+search+API.html, 2026.

[8] Europe PMC. Europe pmc restful web service. https://europepmc.org/RestfulWebService, 2026.

[9] Yichen He, Guanhua Huang, Peiyuan Feng, Yuan Lin, Yuchen Zhang, Hang Li, and Weinan E. Pasa: An llm agent for comprehensive academic paper search. https://arxiv.org/abs/2501.10120, 2025.

[10] Model Context Protocol. Model context protocol. https://modelcontextprotocol.io/, 2026.

[11] OpenAlex. Openalex documentation. https://docs.openalex.org/, 2026.

[12] OurResearch. Unpaywall api. https://unpaywall.org/products/api, 2026.

Table 15: Live academic paper-search anchors. PAPERNEXUS has not yet run PaSa RealScholarQuery or SPARBench, so the external rows define the required future comparison rather than a current score.

System or setting	Reported metrics	Required PAPERNEXUS counterpart
PAPERNEXUS current live diagnostic	ScholarQABench live10: R@10 0.300; precision/F1 not reported	Not the same benchmark; only proves live-provider execution and failure logging.
PaSa-7B	RealScholarQuery: precision 0.5146; recall 0.6111; Recall@20 0.5798	Run the same 50-query gold set with normalized paper identity and provider traces [9].
SPAR	SPARBench: F1 0.3015; precision 0.2932; recall 0.3103	Run SPARBench with the same metric definitions and cache-stamped providers [15].
AutoScholar	SPARBench: **F1 0.3843**; precision 0.3612; **recall 0.4105**	Treat as a stronger agentic-search anchor, not as a current PAPERNEXUS result [15].

Table 16: Method-graph evaluation anchors. These are not retrieval scores; they define the kind of gold graph comparison needed before PAPERNEXUS can make method-evolution quality claims.

System or setting	Reported graph metric	Required PAPERNEXUS counterpart
Intern-Atlas	30 survey-derived graphs: Node Match Ratio 91.0%; Edge Reachable Ratio 89.7%; Path Semantic Correctness 92.0%	Build equivalent gold fixtures, then score canonical method matching, reachable method edges, and path semantics [18].
PAPERNEXUS current graph harness	No public method-graph score yet	Use isolated bench-scratch Kuzu databases and report text-only, graph-only, hybrid retrieval, and method-edge quality.

[13] Semantic Scholar. Semantic scholar academic graph api. https://api.semanticscholar.org/api-docs/, 2026.

[14] Hao Shen, Hang Yang, Zhouhong Gu, and Weili Han. Scholargym: Benchmarking large language model capabilities in the information-gathering stage of deep research. https://arxiv.org/abs/2601.21654, 2026.

[15] Xiaofeng Shi, Yuduo Li, Qian Kou, Longbin Yu, Jinxin Xie, and Hua Zhou. Spar: Scholar paper retrieval with llm-based agents for enhanced academic search. https://arxiv.org/abs/2507.15245, 2025.

[16] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. Beir: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. https://arxiv.org/abs/2104.08663, 2021.

[17] Shirley Wu, Shiyu Zhao, Michihiro Yasunaga, Kexin Huang, Kaidi Cao, Qian Huang, Vassilis N. Ioannidis, Karthik Subbian, James Zou, and Jure Leskovec. Stark: Benchmarking llm retrieval on textual and relational knowledge bases. https://arxiv.org/abs/2404.13207, 2024.

[18] Yujun Wu, Dongxu Zhang, Xinchen Li, Jinhang Xu, Yiling Duan, Yumou Liu, Jiabao Pan, Qiyuan Zhu, Xuanhe Zhou, Jingxuan Wei, Siyuan Li, Jintao Chen, Conghui He, and Cheng Tan. Intern-atlas: A methodological evolution graph as research infrastructure for ai scientists. https://arxiv.org/abs/2604.28158, 2026.

[19] Lei Xiong, Kun Luo, Ziyi Xia, Wenbo Zhang, Jin-Ge Yao, Zheng Liu, Jingying Shao, Jianlyu Chen, Hongjin Qian, Xi Yang, Qian Yu, Hao Li, Chen Yue, Xiaan Du, Yuyang Wang, Yesheng Liu, Haiyu Xu, and Zhicheng Dou. Autoresearchbench: Benchmarking ai agents on complex scientific literature discovery. https://arxiv.org/abs/2604.25256, 2026.

[20] Ruofeng Yang, Yongcan Li, and Shuai Li. Aris: Autonomous research via adversarial multi-agent collaboration. https://arxiv.org/abs/2605.03042, 2026.

[21] Junting Zhou, Wang Li, Yiyan Liao, Nengyuan Zhang, Tingjia Miao, Zhihui Qi, Yuhan Wu, and Tong Yang. Scholarsearch: Benchmarking scholar searching ability of llms. https://arxiv.org/abs/2506.13784, 2025.

Table 17: Operational stage timing from one 10-real-paper local pipeline run. Values are within-run log summaries, not repeated-run estimates.

Measured stage	Unit and n	Observed timing	Statistical status and interpretation
Fast graph commit	10 papers; $n = 1$ run	0.084 s within-run mean	No between-run std, 95% CI, paired test, or effect size; durable writes were small in this observed workload.
Sync	10 papers; $n = 1$ run	0.59 s within-run mean	No between-run std, 95% CI, paired test, or effect size; synchronization was not the observed bottleneck.
LLM optimize	10 papers; $n = 1$ run	113.7 s within-run mean	No between-run std, 95% CI, paired test, or effect size; this was the dominant observed stage.
Active sync	10 papers; $n = 1$ run	120.4 s observed active window	No between-run std, 95% CI, paired test, or effect size; the total active window tracked LLM optimization.

Table 18: Production-style single-paper import timing audit after scoped import processing was enabled. Ranges are observed operational ranges, not confidence intervals.

Stage	Observed timing	Statistical status and operational reading
PDF to Markdown	0.3–1.5 s per paper	Observed range only; no repeated-run std, 95% CI, paired test, or effect size.
Materialize total	0.4–2.2 s per paper	Observed range only; source reading, PDF conversion, Markdown cache, and snapshot creation stayed in low seconds.
LLM Stage 2 semantic extraction	30–40 s per paper	Observed range only; the import task waits primarily on LLM request latency or provider throughput.
Fast commit	3.5–4 s per paper	Observed range only; visible fixed latency but still smaller than LLM extraction in this path.
End-to-end import task	40–46 s per paper	Observed range only; normal LLM-enhanced imports were dominated by Stage 2 in these logs.

A Experiment Run Records

This appendix records the concrete run artifacts behind the measurements in the main text. It is intentionally operational: the entries preserve paths, limits, failure modes, and caveats so that later reruns can distinguish measurement drift from implementation changes. All rows in this appendix remain single observed executions unless explicitly stated. They are therefore audit records rather than repeated-run statistical estimates.

A.1 Execution Environments and Artifact Roots

Table 22 records the two environments used for the local pipeline and public benchmark measurements. The local workstation was used for authenticated upload, queue, and qwen/CPA LLM checks. The 41 server was used for sub-5GB public benchmark runs and the scratch Kuzu import.

The server-side experiment root was:

```
/data1/hyq/papernexus-sub5g-benchmarks/runs/20260512-fixed-corpus-cache
```

The server repo and runtime were:

```
/data1/hyq/PaperNexus
PATH=/data1/hyq/papernexus-runtime/node-v22.16.0-linux-x64/bin:$PATH
```

The long-running production service on the same host was left untouched during the benchmark runs:

```
/data2/hyq/papernexus-runtime/node-current/bin/node ./src/cli/index.js serve --host
    0.0.0.0 --port 4821
```

A.2 Local Pipeline and LLM Audit Records

Table 23 expands the local upload, queue, and pipeline measurements from the main text. Table 24 separately records the qwen batch checks, which used the local OpenAI-compatible `cliproxyapi` configuration with thinking disabled.

The local batch experiment therefore motivates bounded LLM batches with schema validation, output-size budgets, batch-level checkpoints, and split retry/continue. It does not justify a one-prompt-per-corpus strategy.

Table 19: Queue recovery and continue validation from one production-style import incident. Counts describe the audited incident and are not inferential statistics.

Measurement	Before recovery	Recovery evidence	Final active queue state
Historical queue audit	117 completed tasks and 33 failed tasks.	All 33 uploaded source files still existed; 24 were PDF uploads and 9 were Markdown uploads; all 33 had equivalent completed imports later in the queue.	117 completed, 0 failed, 0 pending, 0 running, and overall progress at 100%.

Table 20: Single-prompt multi-paper LLM extraction audit using local `cliproxyapi` with `qwen3.6-plus` and thinking disabled. Each condition was executed once.

Papers in one prompt	Observed completion time	Result	Statistical status and interpretation
1	4.992 s	HTTP 200 with parseable structured output.	$n = 1$; no std, 95% CI, paired test, or effect size; establishes only a one-paper smoke trace.
10	45.561 s	HTTP 200 with parseable structured output.	$n = 1$; no std, 95% CI, paired test, or effect size; small multi-paper batching is feasible in this observed trace.
100	256.409 s	HTTP 200, but the response was truncated at 60,848 characters and JSON parsing failed.	$n = 1$; no std, 95% CI, paired test, or effect size; a 100-paper single prompt is not production-safe.

A.3 Sub-5GB Fixed-Corpus Benchmark Records

Table 25 records the fixed-corpus runtime artifacts executed on the 41 server, and Table 26 records the full-precision metrics. All runs used `-evaluation-mode fixed-corpus`, `-max-candidates 1000`, `-k 1,5,10,20,50,100`, and `PAPERNEXUS_FIXED_CORPUS_SCAN_LIMIT=5000`. ScholarGym additionally required `NODE_OPTIONS=-max-old-space-size=16384`.

The corresponding report paths were:

```
trec-covid-fixed-scan5k/2026-05-12T06-37-46-928Z-trec-covid/report.json
scidocs-fixed-scan5k/2026-05-12T06-45-11-069Z-scidocs/report.json
litsearch-full-fixed-scan5k/2026-05-12T06-49-17-665Z-litsearch/report.json
scholargym-fixed-scan5k-heap16g/2026-05-12T07-49-11-354Z-sage/report.json
```

A.4 Live Provider Diagnostic Records

Table 27 records the live-provider checks. These runs were intentionally treated as diagnostics because provider throttling, query shape, missing gold identifiers, and date-stamped API behavior are part of what they measure.

The report paths were:

```
autoresearchbench-live40/2026-05-12T08-02-34-833Z-sage/report.json
scholarsearch-live10/2026-05-12T08-14-45-762Z-sage/report.json
```

A.5 STaRK-MAG and Kuzu Diagnostic Records

Table 28 records the STaRK-MAG scratch import, and Table 29 records the retrieval diagnostic. The dataset copy lived under `/data1/hyq/papernexus-sub5g-benchmarks/datasets/stark-mag`. Its raw plus extracted footprint was approximately 3.5GB. The extracted graph contained 1,872,968 nodes and 19,919,698 edges.

The STaRK-MAG diagnostic took 400,994ms in total, with average latency 4,773.68ms/query, p50 5,018ms, p95 6,865ms, max 8,135ms, and peak RSS 5.06GB. The report was saved as `stark-mag-kuzu-diagnostic-report.json`. The lexical-only row is a seed baseline, and the graph-expanded row is a one-to-two-hop diagnostic rather than the official STaRK protocol. The

Table 21: Authenticated PDF upload and import stress audit. Each condition used one fresh temporary corpus with one seed Markdown source, so committed paper counts are one larger than the number of uploaded PDFs.

Uploaded PDFs	Request payload	Enqueue latency	End-to-end time	Materialize / PDF parse	Statistical status and committed graph
1	182,972 bytes	14 ms	2.026 s	1.443 s / 1.417 s	$n = 1$; no std, 95% CI, paired test, or effect size; 2 papers, 25 nodes, 55 edges.
100	18,292,646 bytes	60 ms	30.289 s	113.589 s / 110.907 s	$n = 1$; no std, 95% CI, paired test, or effect size; 101 papers, 223 nodes, 1,837 edges.

Table 22: Execution environments used by the reported audits.

Environment	Recorded machine context	Workload role
Local workstation	macOS 15.1 on `arm64`; 10 logical CPUs; 16GB RAM; Node `v25.4.0`.	Authenticated PDF upload, import queue recovery, heuristic-only import stress, and qwen/CPA LLM batching.
41 server	Host `jin-Super-Server`; Linux 6.8.0-90-generic; AMD EPYC 7542 32-Core Processor; 64 logical CPUs; 125GiB RAM; Python 3.10.16; Node `v22.16.0`.	BEIR, LitSearch, ScholarGym, live-provider diagnostics, and STaRK-MAG/Kuzu scratch runs.

graph step improved top-100 candidate coverage but did not improve Hit@10, so ranking still needs relation-aware scoring and high-degree penalties.

The scratch run directory was:

```
/data1/hyq/papernexus-sub5g-benchmarks/runs/20260512-fixed-corpus-cache/stark-mag-
    kuzu-full
```

A.6 Detailed-Record Caveats

The appendix records enough detail to rerun or audit the experiments, but the evidence level is still limited. The fixed-corpus runs use a 5,000-document candidate scan limit and an in-memory per-run index; changing either parameter can change both runtime and recall. The live runs measure provider behavior as much as retrieval logic. The Kuzu diagnostic uses lexical seeds followed by bounded undirected expansion, not an official STaRK-MAG retrieval system. The local LLM experiments use a CPA/qwen deployment whose provider-side hardware, queueing, and batching internals were not visible in local logs. For publication-grade claims, the next version should pin dataset checksums, emit official metric adapters, run at least three independent executions per condition, and report per-query paired tests against fixed baselines.

B Functional Command Sketch

The most common operator path is:

```
papernexus init
papernexus analyze --force
papernexus serve
```

The staged path exposes intermediate state:

```
papernexus materialize --continue
papernexus llm-optimize --continue
papernexus build-graph --continue
papernexus merge-graph --continue
papernexus write-index --continue
```

Retrieval evaluation can be run in fixed-corpus mode:

Table 23: Detailed local audit records for upload, import, queue recovery, and LLM-enhanced pipeline timing.

Run	Scale and setting	Observed record
1-PDF upload/import	1 uploaded PDF; 182,972-byte request payload; one seed Markdown source in the temporary corpus.	Enqueue latency 14ms; end-to-end time 2.026s; materialization 1.443s; PDF parse 1.417s; committed graph contained 2 papers, 25 nodes, and 55 edges. This was a single heuristic-only import trace and excluded external LLM enrichment.
100-PDF upload/import	100 uploaded PDFs; 18,292,646-byte request payload; same seed-source convention.	Enqueue latency 60ms; end-to-end time 30.289s; cumulative materialization 113.589s; cumulative PDF parse 110.907s; committed graph contained 101 papers, 223 nodes, and 1,837 edges. Four-way analysis concurrency explains why cumulative parse time exceeds wall time.
Queue recovery audit	Historical import queue with stale failed tasks and completed replacements.	Before recovery: 117 completed and 33 failed tasks. All 33 source files still existed; 24 were PDF uploads and 9 were Markdown uploads. After reconciliation: 117 completed, 0 failed, 0 pending, 0 running, and progress 100%.
10-paper pipeline log	10 real papers with LLM-enhanced pipeline stages.	Fast graph commit 0.084s within-run mean; sync 0.59s within-run mean; LLM optimize 113.7s within-run mean; active sync window 120.4s. This shows provider-side LLM latency dominated the observed local LLM-enhanced path.

Table 24: Detailed qwen/CPA batch extraction records. Each condition was executed once.

Papers per prompt	Completion time	Outcome
1	4.992s	HTTP 200 with parseable structured output. This is only a smoke trace.
10	45.561s	HTTP 200 with parseable structured output. The result shows that small multi-paper batches are feasible in this local CPA/qwen configuration.
100	256.409s	HTTP 200, but the response was truncated at 60,848 characters and JSON parsing failed. The result shows that a single giant prompt is not production-safe.

```
papernexus benchmark-retrieval ./benchmarks/scifact \
  --format beir \
  --evaluation-mode fixed-corpus \
  --k 1,5,10,20
```

Local secrets can be stored outside plain configuration files:

```
papernexus secure-env set OPENAI_API_KEY
papernexus secure-env list
```

C Remote Agent Pattern

Remote agent access is enabled by running the dashboard server with authenticated streamable HTTP MCP. A client can then call graph lookup, literature discovery, import workflow, and idea-catalyst tools against the same corpus that the local dashboard displays. This pattern is useful when the research corpus stays on one machine but agents or scripts run from another environment.

Table 25: Detailed fixed-corpus runtime records from the 41 server.

Dataset	Scale	Wall / peak RSS	Artifact and caveat
BEIR TREC-COVID	50 queries; 171,332 papers.	8:54.15 / 2.56GB.	Artifact directory `trec-covid-fixed-scan5k`. Large-qrels biomedical regression trace, not a repeated BEIR leaderboard run.
BEIR SciDocs	1,000 queries; 25,657 papers.	6:38.92 / 1.19GB.	Artifact directory `scidocs-fixed-scan5k`. Citation-oriented paper-retrieval regression trace.
LitSearch full	597 queries; 64,183 papers.	2:44.21 / 1.47GB.	Artifact directory `litsearch-full-fixed-scan5k`. Full 597-query run, but not yet the official LitSearch broad/specific metric slices.
ScholarGym	2,458 queries; 570,204 papers.	29:32.76 / 10.25GB.	Static deep-research regression trace; full artifact path is listed below. Not the official ScholarGym leaderboard protocol.

Table 26: Full-precision fixed-corpus metric records. Pool is candidate-pool recall and Zero is zero-match rate.

Dataset	nDCG10	R10	MRR10	Hit100	R100	nDCG100	Pool	Zero
TREC-COVID	0.556899	0.014152	0.870357	1.000000	0.089556	0.380887	0.336728	0.000000
SciDocs	0.152665	0.156750	0.273923	0.763000	0.336283	0.214137	0.541100	0.073000
LitSearch full	0.373935	0.510497	0.333934	0.730318	0.718760	0.416863	0.853099	0.139028
ScholarGym	0.141889	0.188557	0.147330	0.427990	0.330203	0.173783	0.466350	0.420667

Table 27: Detailed live-provider diagnostic records from the 41 server.

Diagnostic	Runtime record	Observed failure mode
AutoResearchBench sample	40 queries; 175 relevant papers; artifact directory `autoresearchbench-live40`; wall 5:24.77; average query latency 8,110.7ms; peak RSS 100,232KB.	Hit@10, R@10, and all retrieval metrics were 0; zero-match rate was 1.000. Provider failures included 58 OpenAlex HTTP 400 responses and 5 Semantic Scholar HTTP 429 responses. The run shows that clue-style tasks need query rewriting/decomposition before retrieval quality can be judged.
ScholarSearch diagnostic	Intended max-10 live run, but the CLI evaluated 50 queries; artifact directory `scholarsearch-live10`; wall 4:14.39; peak RSS 90,300KB.	No usable gold-paper set was available, so ranking metrics are not meaningful. Average retrieved count was 0.26, and the run recorded one Semantic Scholar HTTP 429 response. The run remains a live trace diagnostic only.

Table 28: Detailed STaRK-MAG/Kuzu scratch import records.

Stage	Wall / peak RSS	Observed record
CSV preparation	1:02.87 / 6.91GB.	Wrote import files for 1,872,968 nodes and 19,919,698 edges plus seed lists for 84 human-eval queries. The manifest was saved as `prep-manifest.json`.
Kuzu import	0:10.24 / 2.35GB.	Imported all nodes and edges into a scratch Kuzu database. The scratch artifact footprint was 1.4GB. Pipe-delimited import files were used after comma/quote CSV issues appeared. Summary saved as `kuzu-import-summary.json`.

Table 29: Detailed STaRK-MAG/Kuzu retrieval metric records over 84 human-eval queries.

Method	Hit10	Hit100	R100	nDCG100	MRR
Lexical-only seeds	0.202381	0.345238	0.201853	0.135616	0.153344
Kuzu graph-expanded	0.202381	0.392857	0.255424	0.145688	0.154524